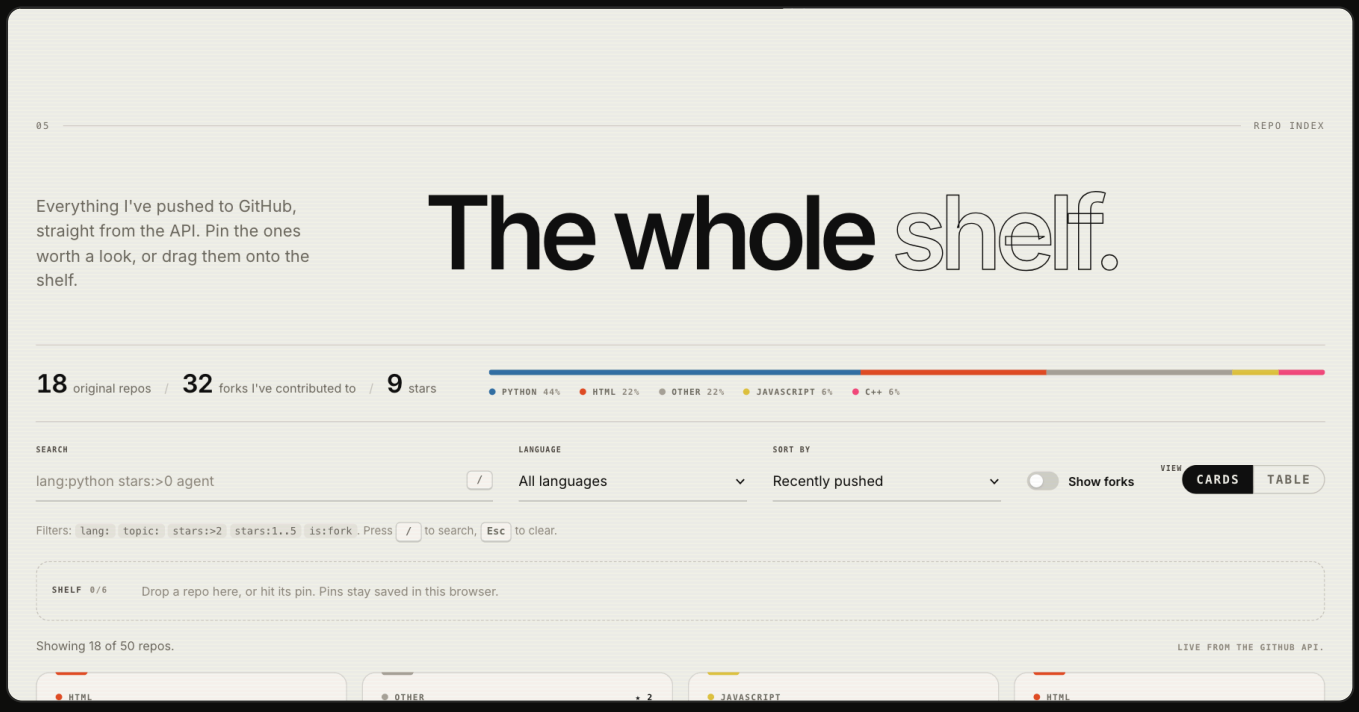
Portfolio2

An interactive personal portfolio built with plain HTML5, CSS3 and JavaScript. It reads my repositories live from GitHub, validates and saves form input in the browser, uses the Geolocation, Drag and Drop and Web Storage APIs, and is deployed on GitHub Pages.



STUDENT	Lukshya Singh Supyal	REGISTRATION NO.	
PROGRAMME	B.Tech CSE, Semester 5	COURSE	CSE326 (L1 T0 P2 · 2 credits)
LIVE WEBSITE	lushadowx.github.io/portfolio2	SOURCE CODE	github.com/LuShadowX/portfolio2

CONTENTS

What's in this report

01 **Project overview**
What it is, tech stack, numbers

02 **How it's built**
Architecture, files, data flow

03 **Tour of the website**
Every section with screenshots

04 **New features in detail**
Repo index, contact form, desk card

05 **Course outcome mapping**
CO1 to CO6 with evidence

06 **Syllabus mapping, unit by unit**
Every topic of Units I–VI

07 **List of practicals**
All 10 experiments with proof

08 **Code walkthrough**
Key snippets, explained

09 **Testing and debugging**
Test cases, error states, devices

10 **Deployment on GitHub Pages**
Repository, commits, hosting

11 **Conclusion**
What I learned

> Every screenshot in this report was taken from the live site at **lushadowx.github.io/portfolio2**. Every code snippet is copied straight from the source files in the repository, so what you read here is exactly what runs.

01 · PROJECT OVERVIEW

A portfolio that does more than sit there

Portfolio2 is my personal website. It introduces me, shows my projects, certificates and open-source work, and lets people get in touch. I built it as a client-side web application: no framework, no build tools, just HTML, CSS and JavaScript running in the browser.

6
JavaScript ES modules added

~1,600
lines of new HTML, CSS and JS

6 / 6
course outcomes covered

10 / 10
lab practicals covered

What the site has

SECTION	WHAT IT DOES	MAIN SYLLABUS AREAS
Intro and hero	Animated title, paper-cutout stickers you can drag, my name and a call to action	HTML5, CSS animations, positioning
Building in public	GitHub contribution heatmap for the last 12 months, drawn as SVG from an API	Fetch, JSON, SVG
About and skills	My story, experience, skills, and a skills "orbit" drawn on canvas	Semantic HTML, Grid, typography
Certificates	A 3D keypad: press a key to open the certificate card, with key sounds	Events, audio, JSON data
Projects, open source, notes	Slide-in catalogue of my work, built from JSON inside the page	Dynamic content, transitions
Repo index *(new)*	All my GitHub repositories, live. Search with filters, card or table view, pin favourites to a shelf by drag and drop.	Fetch, regex, Web Storage, Drag and Drop, tables
Contact *(rebuilt)*	Validated contact form with a saved draft, and a "From the desk" card with live clocks, a map, location distance and a voice note	Forms, regex, Date, Geolocation, iframe, audio

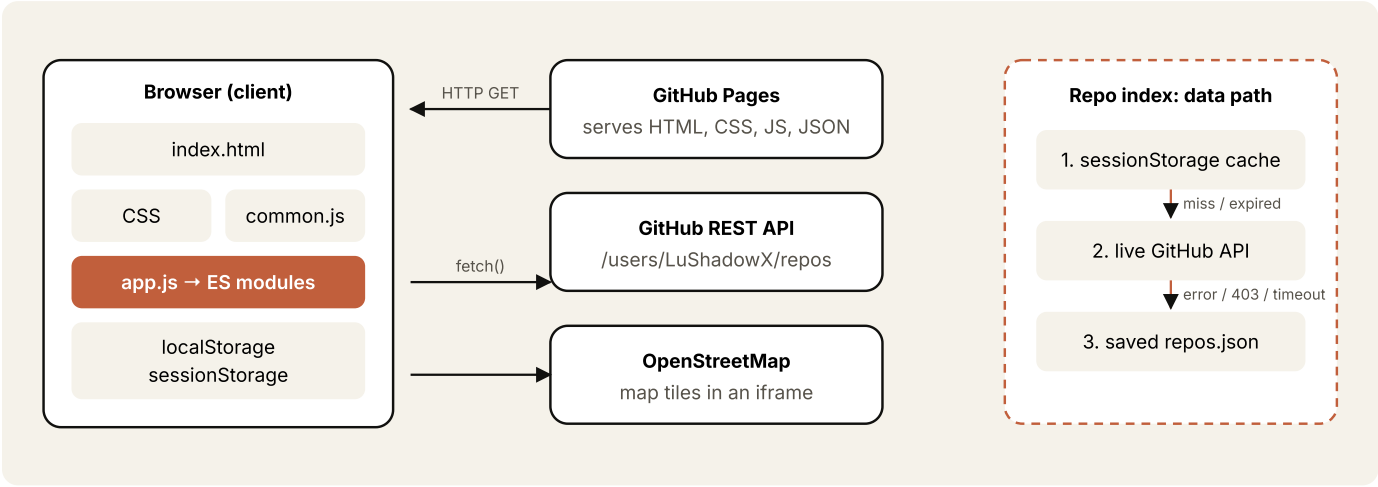
Technology used

Structure	HTML5: semantic elements, forms, table, template, SVG, video, audio, iframe, JSON-LD
Presentation	CSS3: custom properties, Flexbox, Grid, media queries, container queries, `:has()`, pseudo-elements, counters, keyframes, transitions
Behaviour	JavaScript (ES2022), split into ES modules, with no libraries in the new code
Browser APIs	DOM, Fetch, Web Storage, Geolocation, Drag and Drop, HTMLMediaElement, IntersectionObserver, Intl, Performance
External services	GitHub REST API, GitHub contributions API, OpenStreetMap embed
Tools	VS Code, Chrome DevTools, Git, GitHub, GitHub Pages, GitHub Copilot

02 · HOW IT'S BUILT

Architecture

The site follows the client–server model. GitHub Pages is the web server that sends the files. Once the page is in the browser, JavaScript talks to other servers (the GitHub API, OpenStreetMap) to pull in live data.



Files

FILE	JOB	LINES
`index.html`	The page: head metadata, every section, the form, table and card template	443
`assets/css/main.css`	Styles for the whole site, including the new form and desk card	1979
`assets/css/repo-index.css`	Styles for the repo index: controls, cards, shelf, table, container query	272
`assets/js/app.js`	Entry point. Starts each feature safely and loads the repo index lazily.	34
`modules/repo-index.js`	Fetching, search parsing, filtering, sorting, rendering, pins, drag and drop	634
`modules/contact-form.js`	Validation rules, live errors, counter, draft autosave, submit	146
`modules/desk.js`	Clocks and time zones, geolocation distance, voice-note player	152
`modules/storage.js`	Safe wrapper around localStorage and sessionStorage, plus a cache with expiry	45
`modules/utils.js`	Small helpers: `$`, `debounce`, `escapeHTML`, `timeAgo`	38
`modules/debug.js`	`?debug` logging, timings and the error guard	35
`assets/js/common.js`	Existing effects: menu, reveals, heatmap, cursor, certificates, projects	1238
`assets/data/repos.json`	Saved copy of my repositories, used when GitHub can't be reached	data
`robots.txt`, `sitemap.xml`	Search engine instructions and page list	SEO

03 · TOUR OF THE WEBSITE

The page, top to bottom

These are the sections a visitor scrolls through. The first five were already part of my portfolio. They are where most of the HTML and CSS work (Units I–III) lives.



Fig. 1 · **Intro stage.** Animated "Ideas Into Reality" title and paper-cutout stickers that can be dragged with the mouse. Absolute positioning, transforms, keyframe animations, SVG.

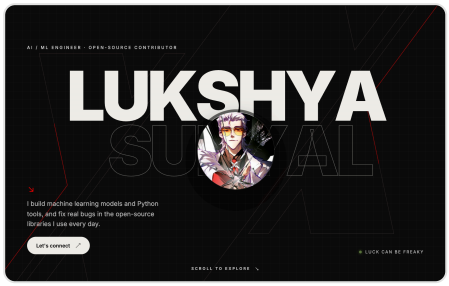


Fig. 2 · **Hero.** Name, role and call to action. Large fluid typography with `clamp()`. The top bar uses Grid and Flexbox.

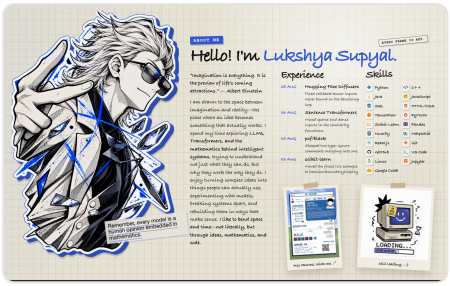


Fig. 3 · **About me.** `article` + `aside` layout, experience and skills lists, images with `alt` text, scroll-reveal transitions.

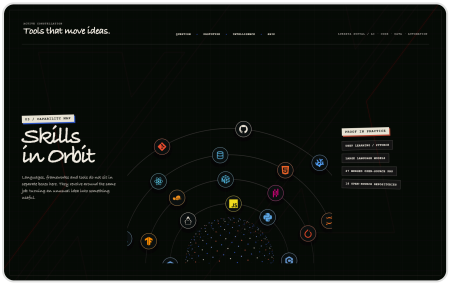


Fig. 4 · **Skills in orbit.** Skill icons placed around an orbit with CSS positioning, and a particle globe on canvas.



Fig. 5 · **Certificates.** A keypad you press to open each certificate. Key sounds are made with the Web Audio API. Certificate data is JSON inside a `script` tag.



Fig. 6 · **Projects.** Catalogue built from JSON at runtime: titles, tags, links and a detail panel.

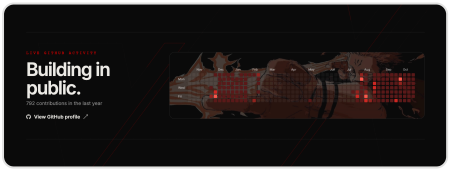


Fig. 7 · **Building in public.** The contribution heatmap is fetched as JSON and drawn as an SVG grid, one `rect` per day, each with a `title` tooltip.

04 · NEW FEATURES IN DETAIL

Repo index

A live list of every repository on my GitHub account, sitting between the projects and the contact section. It's the main client-side application in the project and covers most of Units IV, V and VI.

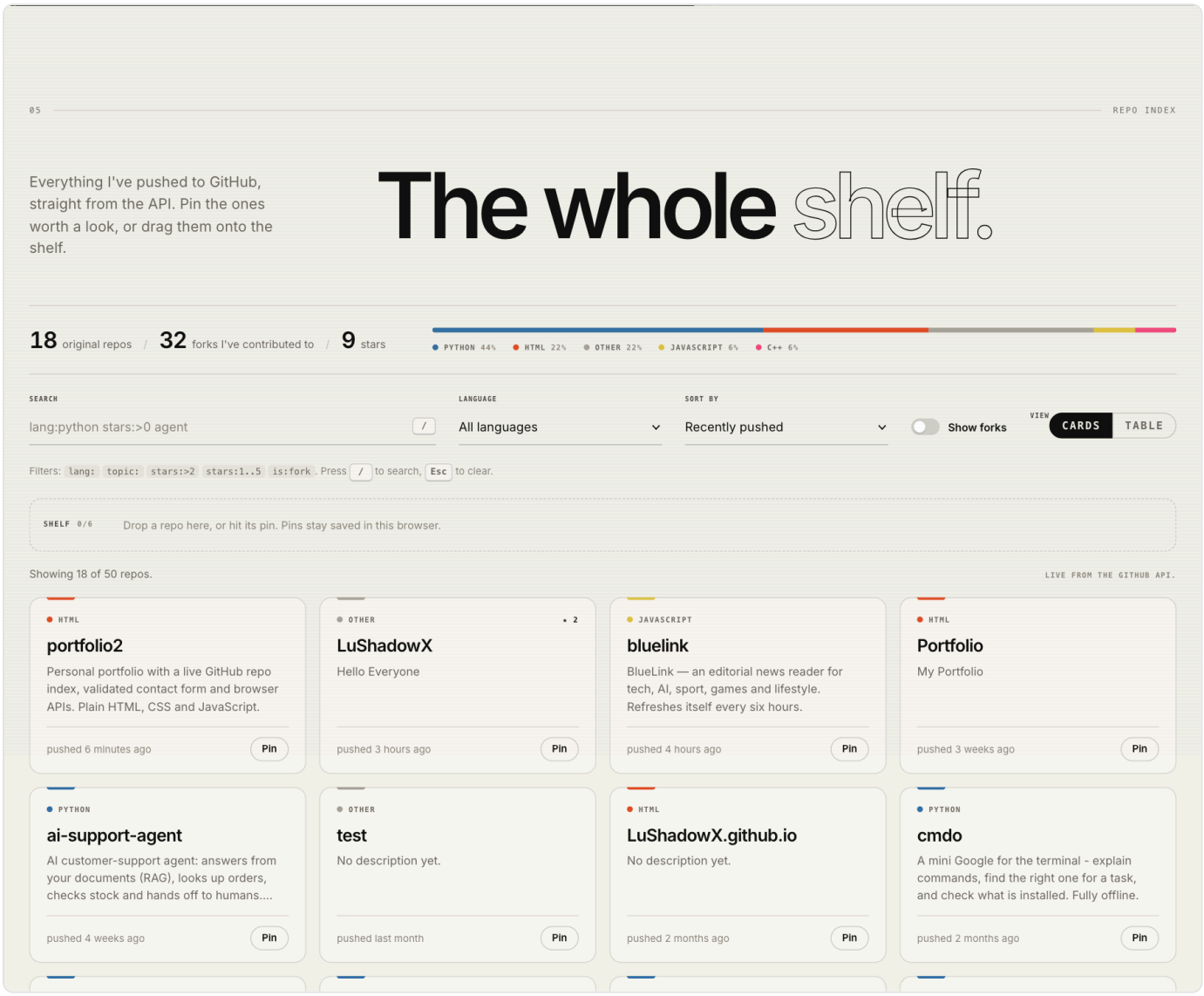


Fig. 8 · **The repo index.** Heading, totals and a language bar, filter controls, the pin shelf, and the repository cards.

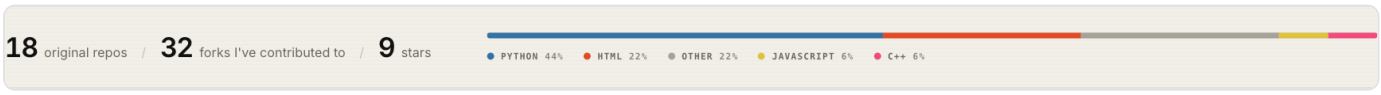


Fig. 9 · **Totals and language bar.** Counts are calculated with `filter` and `reduce`. The coloured bar is an SVG that JavaScript generates from the data.

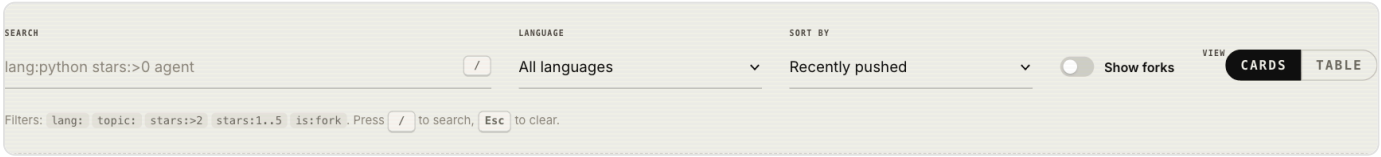


Fig. 10 · **Controls.** Search input, two `select` menus, a switch-style checkbox, and a radio group (`fieldset` + `legend`) for the view. The hint line shows the search filters.

Search with filters

The search box reads GitHub-style filters using regular expressions: `lang:python`, `topic:cli`, `stars:>2`, `stars:1..5`, `is:fork`, plus normal words. If a filter can't be read, it says so instead of failing silently.

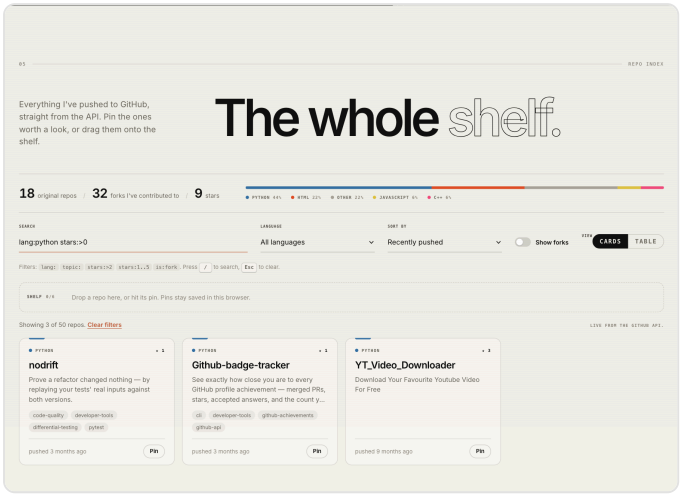


Fig. 11 · `lang:python stars:>0` narrows 49 repos to the 3 Python repos with at least one star.

Fig. 12 · **Bad filter.** `stars:abc` can't be parsed, so the status line explains it and suggests a valid form.

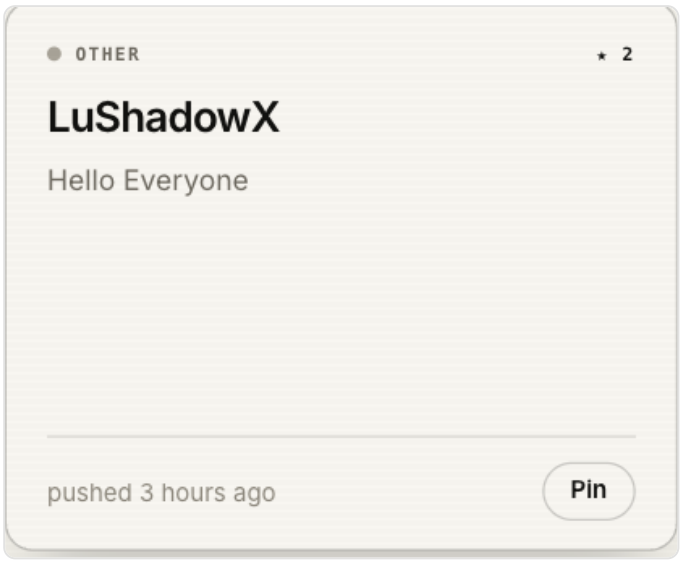


Fig. 13 · **A card.** Language dot, star count, description, "pushed 3 hours ago" (relative time) and a Pin button. On hover the card lifts and the colour bar runs across the top.

Pin shelf (Drag and Drop + localStorage)

Visitors can keep up to six repositories on a shelf. They can drag a card onto it, press its Pin button, or use the keyboard: arrow keys move a focused pin and Delete removes it. The order is saved in `localStorage`, so it's still there after a reload.



Fig. 14 · **Shelf with four pins.** Numbers come from a CSS counter. The second chip has keyboard focus (orange outline).

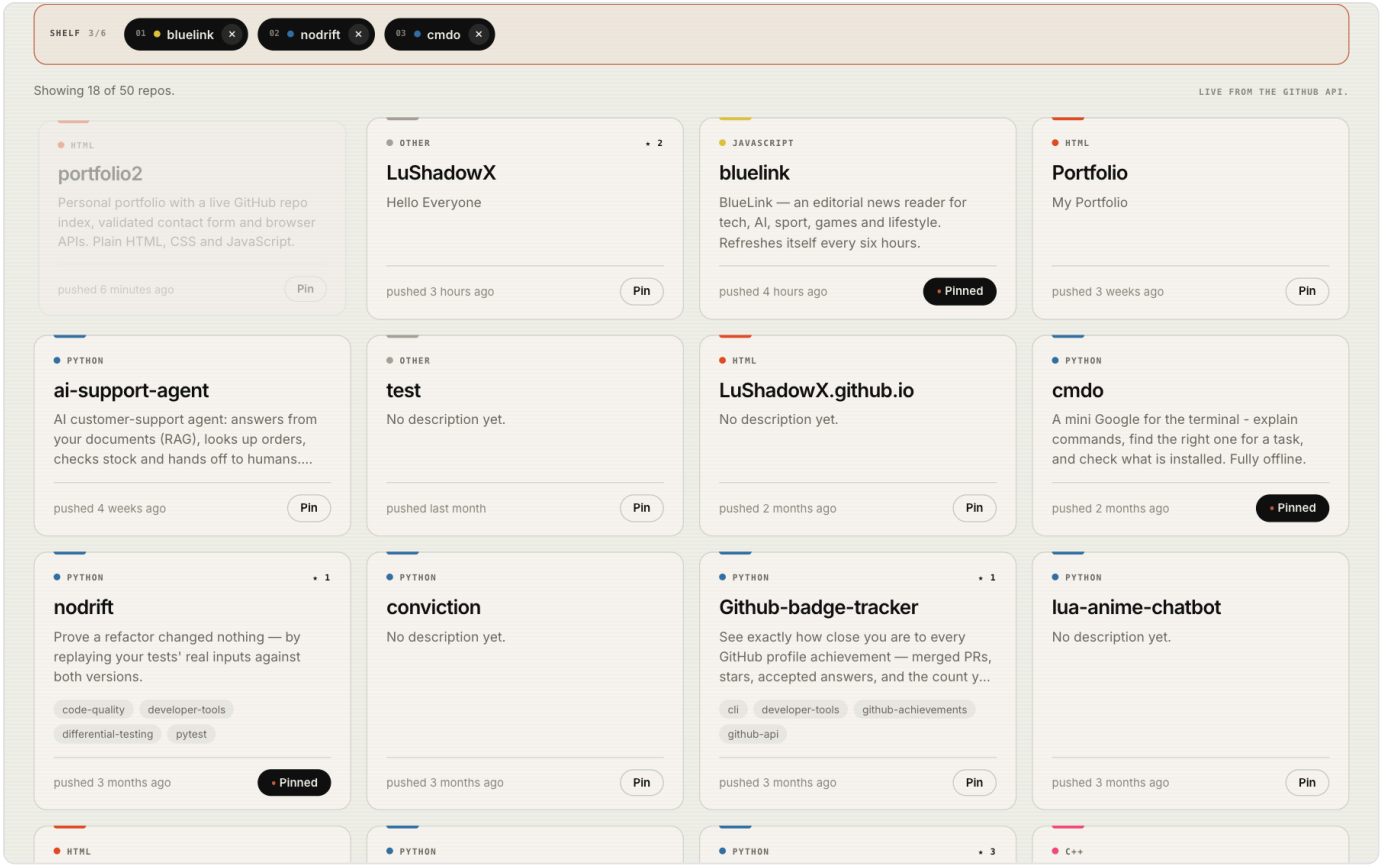


Fig. 15 · **While dragging.** The card being dragged fades, and the shelf switches to a solid highlighted border to show it will accept the drop.

Table view

NAME	LANGUAGE	STARS	UPDATED
YT_Video_Downloader	Python	3	9 months ago
LuShadowX	Other	2	3 hours ago
YT_Video_Downloader_Windows	Other	2	11 months ago
nodrift	Python	1	3 months ago
Github-badge-tracker	Python	1	3 months ago
portfolio2	HTML	0	6 minutes ago
bluelink	JavaScript	0	4 hours ago
Portfolio	HTML	0	3 weeks ago
ai-support-agent	Python	0	4 weeks ago
test	Other	0	last month
LuShadowX.github.io	HTML	0	2 months ago
cmdo	Python	0	2 months ago
conviction	Python	0	3 months ago
lua-anime-chatbot	Python	0	3 months ago
nova-novel-bot	HTML	0	3 months ago
Ai-Limited-Edition-Product-Tracker-	Python	0	3 months ago
Dsa	C++	0	10 months ago
CPP	Other	0	10 months ago

Fig. 16 · **Table view, sorted by stars.** A real `<table>` with `caption`, `thead` and `th scope`. Clicking a header sorts the column and sets `aria-sort`, which the arrow next to "Stars" shows.

04 · NEW FEATURES IN DETAIL

Contact form

The old form only had the browser's built-in checks. The new one validates every field with JavaScript and regular expressions, explains errors in plain words, and never loses what the visitor typed.

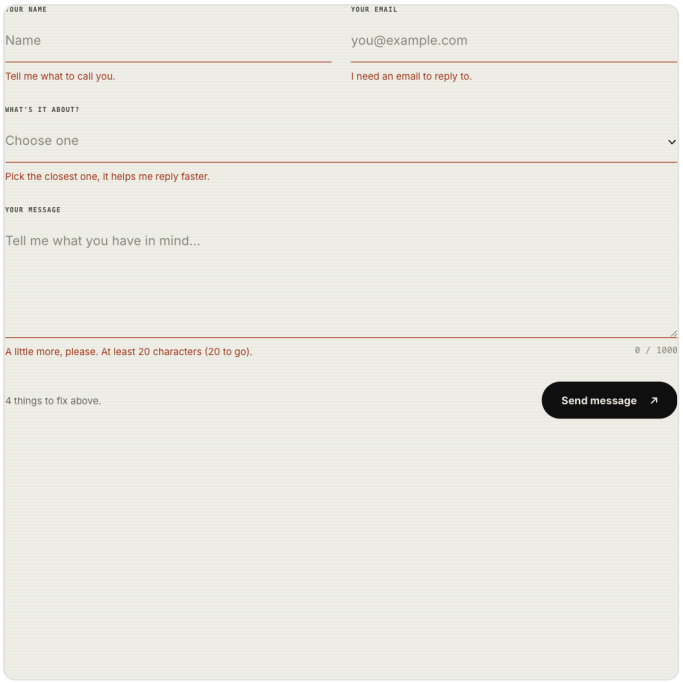


Fig. 17 · **Submit with empty fields.** All four errors appear, each field gets `aria-invalid="true"`, and focus jumps to the first problem.

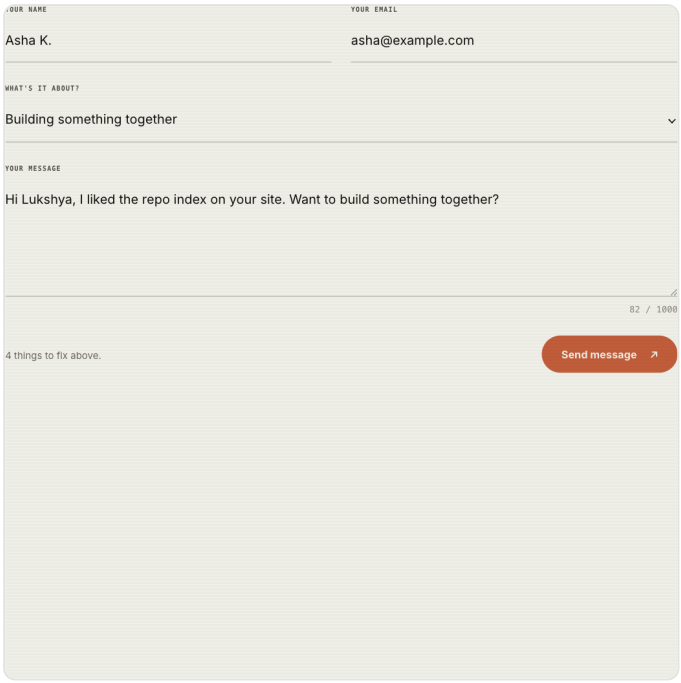


Fig. 18 · **Filled in correctly.** Errors clear while typing. The counter shows characters used out of 1000.

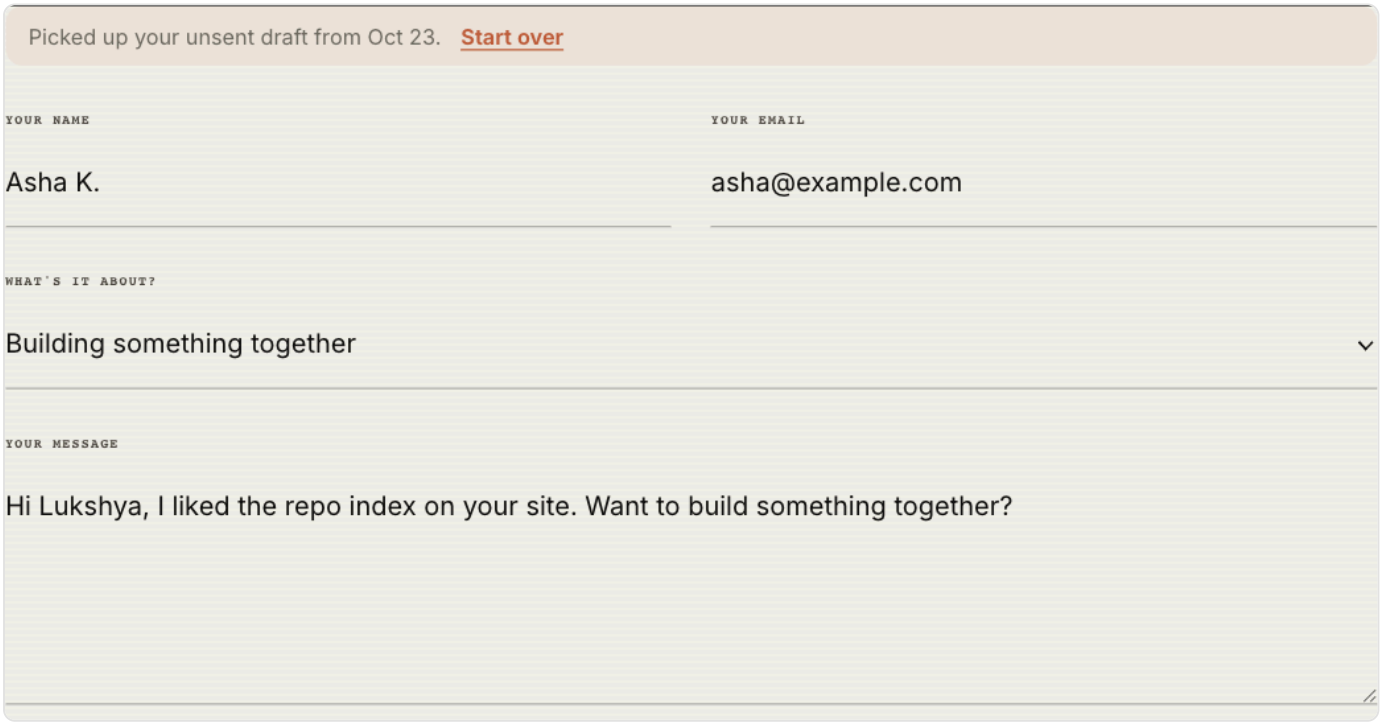


Fig. 19 · **Draft restored after a reload.** The form autosaves to `localStorage` (debounced to 400 ms). On the next visit it fills itself back in and offers "Start over".

FIELD	RULE	MESSAGE SHOWN
Name	Required, 2–60 characters, regex `/^\p{L}[\p{L}\p{M} .'-]{1,59}$/u` (letters from any language)	"Letters, spaces, dots and hyphens only."
Email	Required, regex `/^[^\s@]+@[^\s@]+\.[a-z]{2,}$/i`	"That email doesn't look complete..."
Reason	Required `select`	"Pick the closest one..."
Message	20–1000 characters, at most two links (counted with a URL regex)	"At least 20 characters (n to go)."
Website (hidden)	Honeypot: people never see it, bots fill it in, so the submission is ignored	none

From the desk

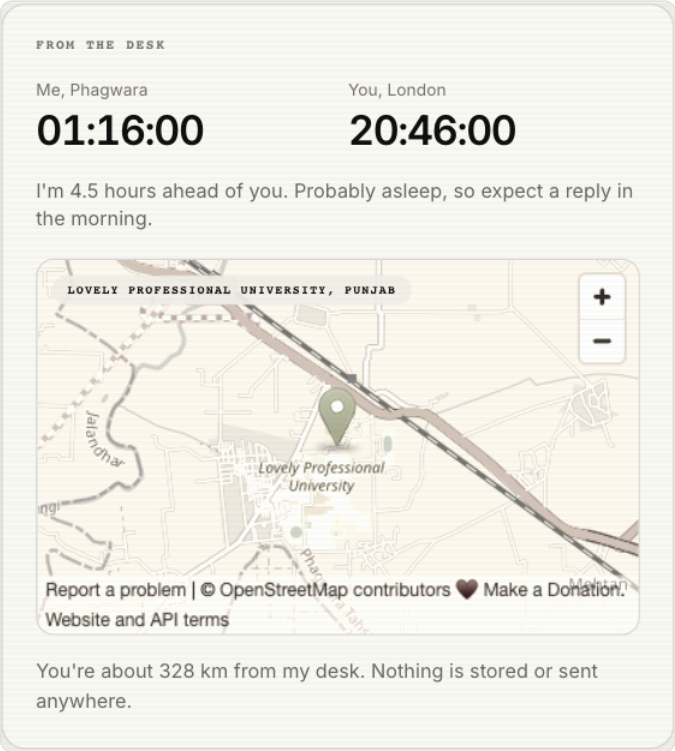


Fig. 20 · **The desk card**, as seen by a visitor in London who allowed location from Delhi.

- **Two live clocks** using `Intl.DateTimeFormat` with time zones: mine (Asia/Kolkata) and the visitor's. They tick exactly on each second.
- **Time-zone gap**: the offset name like "GMT+5:30" is read with a regex and turned into minutes. The note changes with my local hour.
- **Map**: OpenStreetMap inside an `<iframe>` with `title` and `loading="lazy"`.
- **"How far away are you?"** asks for location only on click (Geolocation API), handles every error code, and computes the distance with the haversine formula. Nothing is stored.
- **Voice note**: a custom `<audio>` player with play/pause and a seek bar, driven by media events. It stays hidden until a recording file exists.

Accessibility and mobile

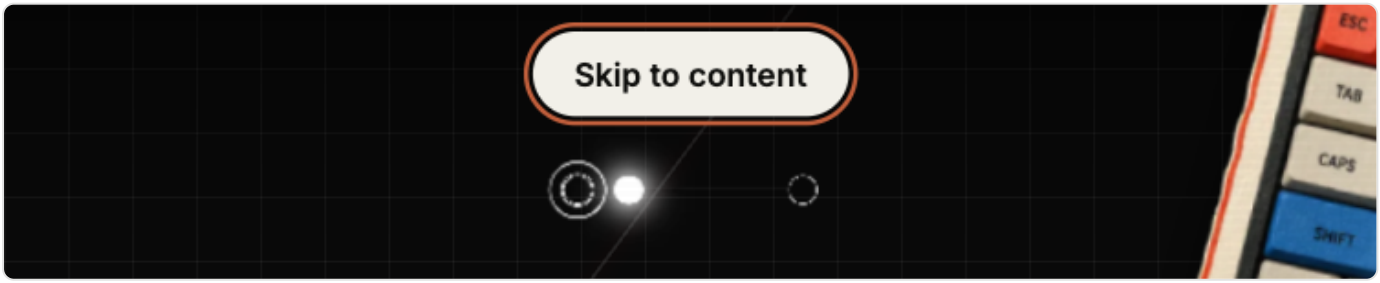


Fig. 21 · **Skip link.** The first Tab press shows "Skip to content", so keyboard users can jump past the intro.

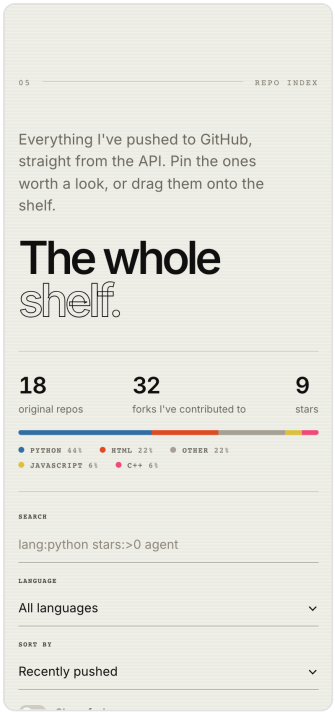


Fig. 22 · Repo index at 390 px.

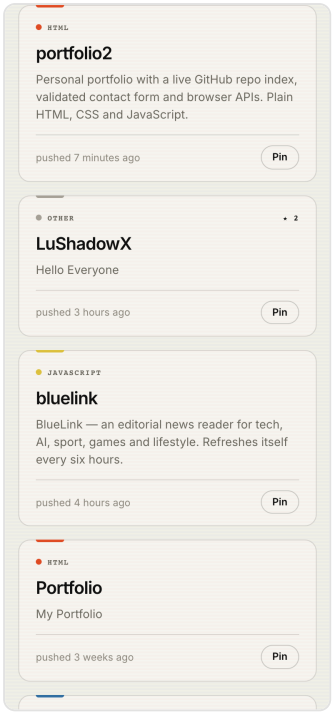


Fig. 23 · Cards in one column.

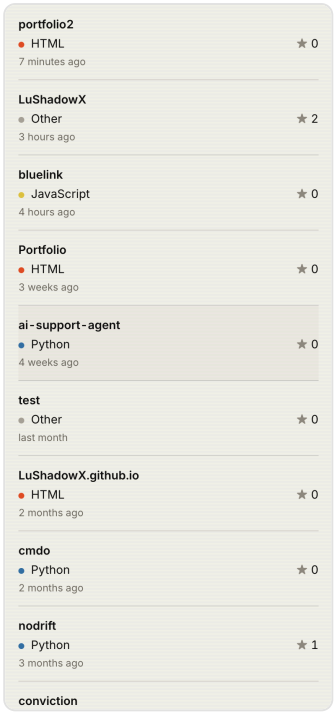


Fig. 24 · Table rows stack (container query).

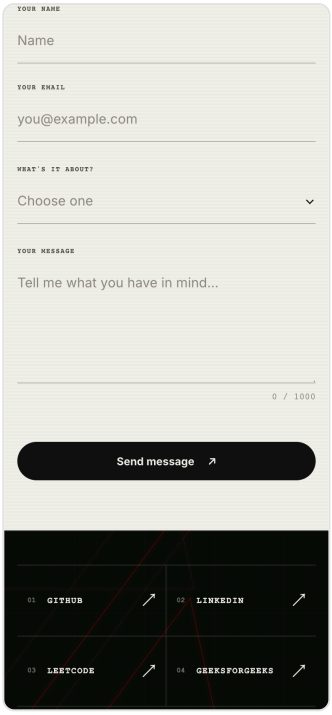


Fig. 25 · Contact form on a phone.

05 · COURSE OUTCOME MAPPING

How each course outcome is met

CSE326 lists six course outcomes. The table shows where each one appears in the project and which figures in this report prove it.

CO1
understand the fundamentals of Internet technologies and HTML5 for web page development

Evidence: Figs 1, 7, 20

CO2
explain the concept of semantic HTML, web accessibility, and web form concepts

Evidence: Figs 10, 16, 28, 21

CO3
apply the CSS techniques to design responsive and visually appealing web interfaces

Evidence: Figs 8, 13, 24

CO4
apply the JavaScript programming concepts to develop dynamic client-side applications

Evidence: Figs 9, 11; Section 08

CO5
demonstrate the interactive web applications using DOM manipulation, events, and browser APIs

Evidence: Figs 27, 15, 19, 20

CO6
analyze and deploy client-side web applications using debugging tools and GitHub Pages

Evidence: Figs 29, 30, 31

CO	HOW THE PROJECT MEETS IT	WHERE
CO1	The client–server model in practice: GitHub Pages serves the files, and the browser calls the GitHub API and gets JSON back. A complete HTML5 document with metadata. Text formatting, lists, links and images. SVG drawn by hand and generated by script. Video, audio and an iframe. SEO through description, canonical, Open Graph, JSON-LD, sitemap and robots.txt.	`index.html`, `sitemap.xml`, `robots.txt`
CO2	Semantic layout (`header`, `nav`, `main`, `section`, `article`, `aside`, `footer`, `figure`, `time`). Accessibility: skip link, labels, live regions, `aria-invalid`, `aria-describedby`, `aria-pressed`, `aria-sort`, and a keyboard alternative to dragging. Forms: search, email, text, select, checkbox, radio, range, textarea. An accessible data table.	`index.html`
CO3	Design tokens as CSS variables. Grid and Flexbox layouts. Media queries at 980, 760 and 480 px, plus a container query. Pseudo-classes and pseudo-elements, attribute selectors, custom form controls, counters, keyframe animations and transitions, and reduced-motion support.	`repo-index.css`, `main.css`
CO4	Constants and variables, every data type, modern operators, string and number methods, conditionals, normal, arrow and async functions, closures, array methods, objects, `Map` and `Set`. Used to build a real application, not separate exercises.	`modules/*.js`
CO5	DOM creation from a `<template>` and with `insertRow`. Click, input, change, keydown, focusout, submit, drag and media events. Regex validation and search. Date and time zones. Web Storage. Fetch and JSON. Geolocation. Drag and Drop.	`repo-index.js`, `contact-form.js`, `desk.js`
CO6	A `?debug` mode with `console.table` and Performance timings. Global error listeners. A guard so one failing feature can't break the page. try/catch/finally with a three-step fallback. Code split into modules. Lazy loading and debouncing for performance. Hosted with GitHub Pages and managed with Git commits.	`app.js`, `debug.js`, GitHub

06 · SYLLABUS MAPPING

Every syllabus topic, unit by unit

Each row is a topic exactly as written in the CSE326 syllabus. The middle column says how the project uses it, and the last column points to the proof.

Unit I: HTML Fundamentals

TOPIC	HOW PORTFOLIO2 USES IT	PROOF
Introduction to Internet and Web Technologies	The site lives on the web at a public URL, uses HTTP(S) and the GitHub REST API, and loads fonts and a map from other servers	§02
Client-Server Architecture	The browser is the client. GitHub Pages and the GitHub API are servers. The code reads status codes (403 rate limit) and reacts to them.	§02, Fig 29
HTML5 Fundamentals	HTML5 doctype, new elements (`template` , `time` , `figure`), new input types, native audio and video	index.html
HTML Document Structure	`html lang` , a `head` with charset, viewport, title, meta and links, then a `body` with semantic sections	§08
HTML Elements and Attributes	Global attributes (`id` , `class` , `hidden` , `tabindex`), `data-*` attributes as JS hooks, `draggable` , `loading` , `rel`	index.html
Text Formatting	`strong` , `em` , `b` , `small` , `code` , `kbd` (keyboard hints)	Fig 10
Lists	`ol` for the ordered pin shelf, `ul` for cards, topics and the language legend, `dl` for the clocks	Figs 27, 20
Hyperlinks and Images	Nav links, in-page anchors, external links with `target` and `rel="noopener"` , images with `alt` , `width` / `height` and lazy loading	Figs 2, 3
SVG Graphics	Inline SVG icons, the heatmap (one `rect` per day), and the language bar generated from data	Figs 7, 9
Multimedia Elements Audio, Video and Iframes	`<audio>` voice note with two sources, `<video>` menu previews, OpenStreetMap `<iframe>`	Fig 20
SEO fundamentals	Title, meta description, canonical URL, Open Graph and Twitter tags, JSON-LD `Person` schema, sitemap.xml, robots.txt	§08
HTML Best Practices	Semantic tags over `div` s, every image has `alt` , every input has a label, no inline event handlers in the new code, a `noscript` message	index.html

Unit II: Semantic HTML and Forms

TOPIC	HOW PORTFOLIO2 USES IT	PROOF
Semantic HTML Elements	`header` , `nav` , `main` , `section` , `article` , `aside` , `footer` , `figure` / `figcaption` , `time datetime`	§08
Page Structure and Layout	Each section is labelled with `aria-labelledby` . Headings run in order. Each card is an `article` with a `footer` .	Fig 8
Accessibility Fundamentals	Skip link, visible focus outlines, live regions announce results and pin changes, buttons have text or `aria-label`	Figs 21, 27
Forms and Form Controls	Contact form plus the filter form (`role="search"`): input, select, textarea, checkbox, radio, range, button	Figs 10, 28

TOPIC	HOW PORTFOLIO2 USES IT	PROOF
Input Types and Attributes	`search`, `email`, `text`, `checkbox`, `radio`, `range`. `required`, `minlength`, `maxlength`, `autocomplete`, `inputmode`, `placeholder`, `spellcheck`.	§08
Form Validation Basics	HTML constraints are in the markup. `novalidate` hands control to JavaScript for clearer messages.	Fig 28
Tables and Data Representation	Repo table with `caption`, `thead` / `tbody`, `th scope="col"` / `scope="row"`, and sortable headers	Fig 16
Accessibility Standards	WCAG-style practice: `aria-invalid` + `aria-describedby` for errors, `aria-sort`, `aria-pressed`, `aria-live`, `prefers-reduced-motion`, everything usable by keyboard	Figs 28, 16
Responsive Content Structuring	Content order makes sense in one column. Controls go from 5 columns to 2 to 1. The table turns into stacked rows.	Figs 22 to 25

Unit III: Cascading Style Sheets

TOPIC	HOW PORTFOLIO2 USES IT	PROOF
Introduction to CSS	Two external stylesheets linked in the head, organised by section, with a cache-busting version query	§02
CSS Selectors and Specificity	Low-specificity class selectors with BEM-style names (`.repo-card__name`), state classes (`.is-over`), descendant and child selectors	`repo-index.css`
Colors and Backgrounds	Hex colours with alpha (`#11110f24`), gradients for the shimmer and seek bar, an SVG data-URI as the select arrow	Fig 10
Units and Measurements	`px` , `rem` , `em` , `%` , `vw` , `mm` (print), and `clamp()` for fluid headings	`repo-index.css`
Typography	Inter for text, a monospace face for labels, tight letter-spacing on headings, `font-variant-numeric: tabular-nums` for clocks and counts	Figs 8, 20
Text Styling	Uppercase spaced labels, outlined heading text (`-webkit-text-stroke`), animated underline, line clamping to three lines	Fig 13
Box Model	`box-sizing: border-box` , padding, borders and radius on cards, chips and the shelf	Fig 13
Positioning Techniques	Relative cards with an absolute colour bar, the absolute `kbd` hint, the drop marker, the fixed skip link, the sticky top bar	Figs 15, 21
Flexbox Layout	Shelf chips, card footers, status line, desk distance row, voice player	Fig 27
CSS Grid Layout	Card list (`repeat(auto-fill, minmax(min(100%, 280px), 1fr))`), the controls row, the heading layout, the clocks	Fig 8
Pseudo Classes	`:hover` , `:focus` , `:focus-visible` , `:checked` , `:invalid` , `:disabled` , `:empty` , `:has()` , `:not()` , `:first-of-type` , `:placeholder-shown`	§08
Pseudo Elements	`::before` and `::after` for language dots, counter numbers, sort arrows and the switch knob. `::placeholder` , range thumbs.	Figs 27, 16
Attribute Selectors	`[data-fork="true"]` , `[aria-pressed="true"]` , `[aria-sort="ascending"]` , `[data-state="error"]` , `[data-label]` , `[hidden]`	§08
Form Styling	Underlined inputs, a custom select arrow, a switch-style checkbox, segmented radio buttons, a styled range slider, red error states	Figs 10, 28
CSS Variables	Section tokens (`--ink` , `--paper` , `--accent`) and per-item variables set from JS (`--lang` , `--progress` , `--dot`)	§08
Responsive Web Design	Media queries at 980, 760 and 480 px, `hover: none` for touch screens, and a container query on the results	Figs 22 to 25
CSS Animations	`@keyframes repo-shimmer` (loading cards), `shelf-nope` (shake when the shelf is full), plus the site's intro animations	§08
CSS Transitions	Card lift, colour bar growth, underline, switch knob, chip spacing during drag, button colours	Fig 13

Unit IV: JavaScript Fundamentals

TOPIC	HOW PORTFOLIO2 USES IT	PROOF
Introduction to JavaScript	Loaded as ES modules (`type="module"`), running after the HTML is parsed	§02
Variables and Constants	`const` by default (settings such as `MAX_PINS` , `USER`), `let` only where a value changes (`dragging` , `timer`)	§08
Data Types	string, number, boolean, null, undefined, objects, arrays, `Date` , `Map` , `Set` , regular expressions	§08
Operators and Expressions	`??` , `?.` , `||` / `&&` short-circuit, spread, `**` in the distance formula, comparison and arithmetic operators	§08
Strings and String Methods	Template literals, `replace` , `split` , `includes` , `toLowerCase` , `padStart` , `trim` , `startsWith`	`repo-index.js`
Numbers and Boolean Values	`Math.round` / `sin` / `cos` / `asin` , `toFixed` , `Number.isInteger` , `Intl.NumberFormat` , Boolean flags (`loaded` , `reverse`)	`desk.js`
Conditional Statements	`if/else` , `switch` (filters, arrow keys), ternary operators	§08
Functions	Function declarations, arrow functions, async functions, default parameters, rest parameters, functions returning functions	§08
Scope	Module scope keeps helpers private. Block scope with `let` / `const` . Closures in `debounce` , `cache` and `wrap` .	§08
Arrays and Array Operations	`map` , `filter` , `reduce` , `sort` , `find` , `findIndex` , `some` , `every` , `splice` , `slice` , `includes` , `Array.from`	§08
Objects and Object Manipulation	The `state` object, destructuring, spread to merge saved settings, `Object.entries` / `fromEntries` , a lookup object of sort functions	§08

Unit V: Interactive Web Development

TOPIC	HOW PORTFOLIO2 USES IT	PROOF
Loops and Iterative Programming	`for...of` in `timeAgo`, `forEach` over results, iterating a `Map` to build menu options	§08
Higher Order Functions	A filter → filter → filter → sort pipeline, `debounce` (takes and returns a function), `guard` (runs a function safely)	§08
Callback Functions	Event listener callbacks, sort comparators, the Geolocation callbacks wrapped in a Promise, the `replace` callback in the query parser	§08
DOM Fundamentals	The page is read and changed through the DOM tree. The `<template>` content is cloned for each card.	§08
DOM Selection and Manipulation	`querySelector(All)` helpers, `closest`, `classList`, `dataset`, `setAttribute`, `replaceChildren`, `insertRow` / `insertCell`, `DocumentFragment`	Figs 8, 16
Event Handling	click, input, change, submit, keydown, focusout, dragstart/dragover/dragleave/drop/dragend, media events. Event delegation on lists.	§08
Dynamic Content Generation	Cards, table rows, pins, the language menu, totals and the SVG bar are all built from data at runtime	Figs 8, 9
JavaScript Accessibility Practices	Focus goes back to the button after re-render and moves to the first invalid field on submit. Live-region messages. Arrow and Delete keys for pins.	Figs 27, 28
Form Validation	A `RULES` object of validator functions. Errors show after leaving a field, then update live while typing.	Figs 28, 26
Regular Expressions	Name and email patterns, link counting, the search-filter parser with capture groups, the number-range rule, the GMT offset parser	Figs 11, 12
Date and Time Handling	`Date`, `Intl.DateTimeFormat` with time zones, `Intl.RelativeTimeFormat` ("3 days ago"), a clock that ticks on the second	Figs 13, 20
Audio and Video Events	Voice note: `loadedmetadata`, `timeupdate`, `play`, `pause`, `ended`, `error`. Menu videos: `playing`, `waiting`.	§08
Browser Storage using Web Storage API	localStorage: pins, view settings, form draft. sessionStorage: 30-minute API cache with an expiry check.	Figs 27, 19
Fetch API and JSON Processing	`fetch` + `res.json()`, a `res.ok` check, an `AbortController` timeout, fallback to a local JSON file	Figs 8, 29
Geolocation API	`getCurrentPosition` on click only, with timeout and maxAge options, every error code handled, haversine distance	Fig 20
Drag and Drop API	`draggable` cards and chips, `dataTransfer`, a drop position worked out from the pointer, dragging a chip off the shelf unpins it	Figs 27, 15

Unit VI: Web Application Development and Deployment

TOPIC	HOW PORTFOLIO2 USES IT	PROOF
Debugging Techniques	Add `?debug` to the URL to see timings, a `console.table` of repos and warnings with stack traces	§09

TOPIC	HOW PORTFOLIO2 USES IT	PROOF
Browser Developer Tools	Network panel (offline and throttled tests, 403 responses), Application panel (storage keys), Console, device mode for phones	§09
Error Handling	try/catch/finally, errors carrying extra data (`rateLimited`), a three-step fallback, a retry button, global `error` and `unhandledrejection` listeners	Figs 29, 30
Interactive User Interface Development	Search as you type, toggles, sorting, drag and drop, live clocks, inline validation	§04
Client-Side Application Design	One `state` object, pure helpers (`parseQuery` , `normalize`), and render functions that rebuild the view from state	§08
Code Organization	ES modules: an entry file, three feature modules and three shared helper modules	§02
Website Performance Fundamentals	The API is called only when the section comes near (IntersectionObserver). Debounced search, batched DOM updates, preconnect, lazy iframe and images, session cache.	§08
GitHub Repository Management	Public repository, small focused commits with clear messages, README with setup and structure, description and website link set	Figs 31, 32
Website Hosting using GitHub Pages	Deployed from the `main` branch root. Every push redeploys automatically.	§10
Web Development Best Practices	Escaping data before inserting HTML, no secrets in code, graceful failure, accessibility, reduced motion, consistent code style	§08
Improving Web Applications using GitHub Copilot	Used Copilot suggestions to review code and spot edge cases, then checked every change myself in DevTools	§09

All 10 lab experiments, inside one project

Each experiment from the CSE326 practical list is part of a working feature. Here's where to find each one.

P1 Design a user registration form using HTML input controls and apply CSS selectors, the box model, and focus styles

The contact form uses text, email, select and textarea controls with labels. CSS selectors target `.field`, `.has-error` and `:focus`. The box model shows in the padding and bottom borders. Focus turns the border orange, and buttons get a `:focus-visible` outline.

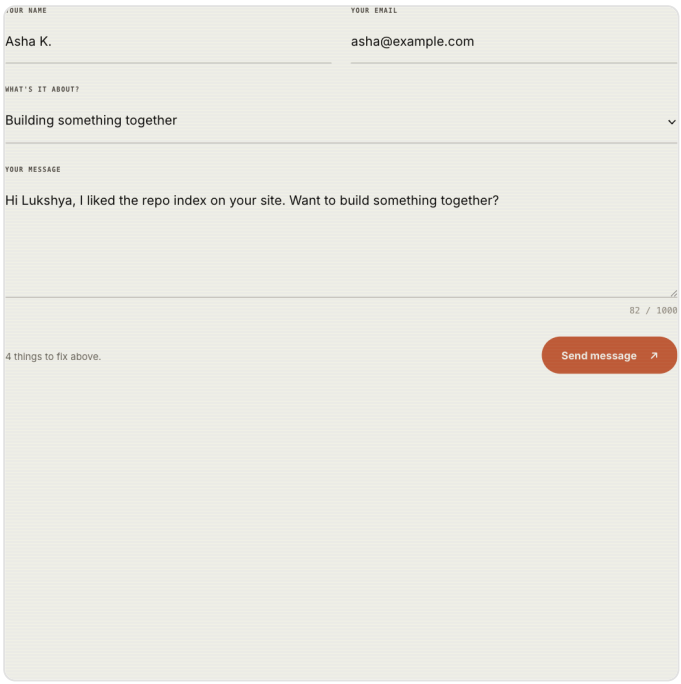


Fig. 26 · Contact form with focus and valid states.

P2 Create a web page that includes text formatting, lists, hyperlinks, images, SVG graphics, audio, video, and an iframe using HTML5 elements

All on `index.html`: formatting (`strong`, `em`, `kbd`, `code`), lists (`ol`, `ul`, `dl`), links and images throughout, SVG (heatmap, icons, language bar), `<video>` in the menu, `<audio>` voice note and an OpenStreetMap `<iframe>` (Fig 20).

P3 Build a responsive web page using Flexbox for navigation, CSS Grid for content layout, and media queries for different screen sizes

The top navigation is a Grid bar with Flexbox link groups. The repo cards use a CSS Grid that fills the width. Media queries reshape the controls at 980, 760 and 480 px, and a container query restacks the table (Figs 8, 22 to 25).

P4 Design a responsive webpage using CSS variables, pseudo-classes, pseudo-elements, transitions, and animations to enhance user interaction

The variables `--ink`, `--paper`, `--accent` and the per-card `--lang`. Pseudo-classes such as `:has()`, `:checked` and `:focus-visible`. Pseudo-elements for dots, numbers and arrows. The card lift and colour-bar transitions, plus the shimmer and shake keyframes (Fig 13; CSS in §08).

P5 Write JavaScript programs demonstrating variables, constants, data types, operators, expressions, and control flow statements

`parseQuery` and `starsRule` use constants, regex, string and number types, comparison operators, a `switch` and ternaries. `moodAt` in `desk.js` is a clear `if` chain. See §08.

P6 Implement JavaScript functions (normal and arrow functions) and perform operations on arrays and objects

Normal functions: `parseQuery`, `offsetMinutes`, `distanceKm`. Arrow functions: `normalize`, `debounce`, the sort lookup `SORTS`. Arrays: the filter/sort pipeline in `visibleRepos`. Objects: `state`, destructuring, spread (§08).

P7 Handle user events such as clicks, keyboard input, and form changes to dynamically update web page content using JavaScript and DOM manipulation

Clicks: Pin buttons, table headers, the view switch. Keyboard: `/` to search, `Esc` to clear, arrow keys and Delete on pins. Form changes: language, sort and fork filters re-render the list straight away.



Fig. 27 · Pins added by click, drag and keyboard.

P8 Develop a form validation system using JavaScript and regular expressions to validate user input before form submission

The `RULES` object in `contact-form.js` has a regex or rule for each field. `check()` sets `aria-invalid` and the message. Submit is blocked until every field passes (Figs 28, 26).

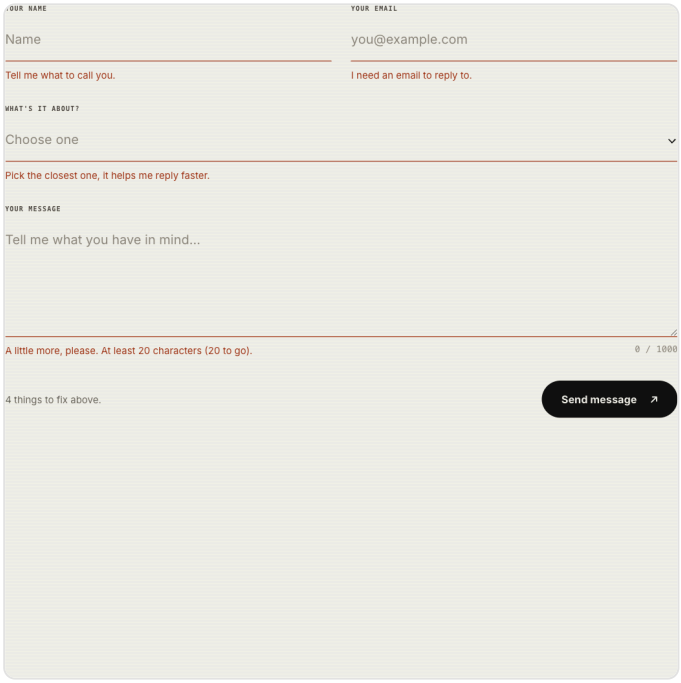


Fig. 28 · Validation blocking the submit.

P9 Demonstrate JavaScript execution context, scope, closures, loops, callback functions, and higher-order functions using suitable examples

`debounce` keeps `timer` in a closure. `cache(key, ttl)` returns functions that remember their key. Module scope hides helpers. `for...of` in `timeAgo`. The callback-based Geolocation API is wrapped in a Promise in `locate`. Array pipelines are higher-order functions (§08).

P10 Create an HTML5 web page using semantic elements and basic CSS for layout and typography

The whole site: `header`, `nav`, `main`, `section`, `article`, `aside`, `footer`, with Inter and a monospace font for typography (Section 03).

08 · CODE WALKTHROUGH

The important code, explained

Short extracts from the real source files, each with what it does and which syllabus topics it shows.

1. Fetching with a timeout and a fallback

repo-index.js · lines 65-79

```
async function fetchJSON(url, timeout = 8000) {
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(),
timeout);
    try {
        const res = await fetch(url, { signal:
controller.signal });
        if (!res.ok) {
            const err = new Error(`${res.status}
${res.statusText} for ${url}`);
            err.rateLimited = res.status === 403 &&
res.headers.get('x-ratelimit-remaining') === '0';
            throw err;
        }
        return await res.json();
    } finally {
        clearTimeout(timer);
    }
}
```

`fetchJSON` wraps `fetch`. An `AbortController` cancels the request after 8 seconds. A non-OK response becomes an `Error` carrying a `rateLimited` flag, read from GitHub's rate-limit header.

`finally` clears the timer whether the request worked or not.

`Fetch API` `JSON` `async/await` `Error handling`

repo-index.js · lines 82-100

```
async function loadRepos({ fresh = false } = {}) {
    const cached = fresh ? null : repoCache.read();
    if (cached) return { repos: cached, source: 'cache'
};

    try {
        const live = (await fetchJSON(API)).map(trim);
        repoCache.write(live);
        return { repos: live, source: 'live' };
    } catch (err) {
        debug.warn('GitHub API unavailable, falling back
to the snapshot.', err);
        const snapshot = await fetchJSON(SNAPSHOT);
        return {
            repos: snapshot.repos,
            source: 'snapshot',
            savedAt: new Date(snapshot.savedAt),
            reason: err.rateLimited ? 'rate-limit' :
err.name === 'AbortError' ? 'timeout' : 'offline'
        };
    }
}
```

Three steps: the session cache first, then the live API, then the copy committed with the site. The result says which one was used, so the page can tell the visitor honestly (Fig 29).

`Client-server` `sessionStorage` `try/catch`

2. Parsing search filters with regular expressions

repo-index.js · lines 104–107

```
// Plain words plus a few GitHub-style qualifiers:
//   lang:python  topic:ml  stars:>2  stars:1..5
is:fork  is:source
const QUALIFIER = /\b(lang|language|topic|stars|is):("
[^"]*"|\S+)/gi;
const NUMBER_RULE = /^(?:(≥|<=▷|<)?(\d+)|(\d+)\.\.
(\d+))$/;
```

`QUALIFIER` finds every `key:value` pair. Group 1 is the key, and group 2 is the value (a quoted phrase or a single word). The `g` and `i` flags match all of them, in any case.

`NUMBER_RULE` accepts `5`, `>2`, `⩽10` or a range `1..5`.

repo-index.js · lines 124–159

```
export function parseQuery(input) {
    const tests = [];
    const problems = [];
    let forkMode = null;

    const rest = input.replace(QUALIFIER, (token, key,
raw) ⇒ {
        const value = raw.replace(/"/g,
'').toLowerCase();
        switch (key.toLowerCase()) {
            case 'lang':
            case 'language':
                tests.push(repo ⇒ (repo.language ??
'other').toLowerCase() ⩵⩵ value);
                break;
            case 'topic':
                tests.push(repo ⇒
repo.topics.some(topic ⇒ topic.includes(value)));
                break;
            case 'stars': {
                const rule = starsRule(value);
                if (rule) tests.push(repo ⇒
rule(repo.stars));
                else problems.push(token);
                break;
            }
            case 'is':
                if (value ⩵⩵ 'fork') forkMode = 'fork';
                else if (value ⩵⩵ 'source') forkMode =
'source';
                else problems.push(token);
                break;
        }
        return ' ';
    });


rest.toLowerCase().split(/\s+/).filter(Boolean).forEach(wor
⇒ {
        tests.push(repo ⇒ `${repo.name}
${repo.description} ${repo.topics.join('
')}`.toLowerCase().includes(word));
    });

    return { tests, problems, forkMode };
}
```

`replace` with a callback visits every filter. A `switch` turns each one into a test function, and anything it can't read goes into `problems` (Fig 12). Whatever text is left becomes plain-word searches.

Regular expressions · switch · Callbacks · Strings

3. Higher-order functions: filter, sort and reduce

repo-index.js · lines 208-217

```
function visibleRepos() {
    const { tests, problems, forkMode } =
parseQuery(state.query);
    const compare = SORTS[state.sort];
    const list = state.repos
        .filter(repo => forkMode === 'fork' ? repo.fork
: forkMode === 'source' ? !repo.fork : state.forks ||
!repo.fork)
        .filter(repo => state.language === 'all' ||
(repo.language ?? 'Other') === state.language)
        .filter(repo => tests.every(test => test(repo)))
        .sort((a, b) => state.reverse ? compare(b, a) :
compare(a, b));
    return { list, problems };
}
```

Every visible list is computed from `state`. Three `filter` calls handle the forks switch, the language menu and the search tests. `sort` uses a comparator from the `SORTS` lookup object, reversed when a table header is clicked twice.

`Arrays` `Higher-order functions` `Destructuring`

4. Drag and Drop

repo-index.js · lines 542-550

```
section.addEventListener('dragstart', e => {
    const item = e.target.closest?.('.repo-card, .repo-
chip');
    if (!item) return;
    dragging = { name: item.dataset.name, from:
item.classList.contains('repo-chip') ? 'shelf' : 'list'
};
    e.dataTransfer.effectAllowed = 'move';
    e.dataTransfer.setData('text/plain', dragging.name);
    requestAnimationFrame(() => item.classList.add('is-
dragging'));
    section.classList.add('is-dragging');
});
```

One listener on the whole section handles every card and chip (event delegation). It remembers what is being dragged and where from, puts the name in `dataTransfer`, and adds CSS classes for the visual feedback in Fig 15.

repo-index.js · lines 572-577

```
el.shelf.addEventListener('drop', e => {
    e.preventDefault();
    const name = e.dataTransfer.getData('text/plain') ||
dragging?.name;
    if (name) pinAt(name, dropIndex(e.clientX,
e.clientY));
    clearMarkers();
});
```

On drop, `dropIndex` works out where the pointer is between chips, and `pinAt` inserts the repo there and saves the order to `localStorage`.

`Drag and Drop API` `Events` `localStorage`

5. Web Storage with closures

storage.js · lines 5-31

```
const wrap = (name) => {
    const area = () => window[name];
    return {
        get(key, fallback = null) {
            try {
                const raw = area().getItem(key);
                return raw === null ? fallback :
JSON.parse(raw);
            } catch {
                return fallback;
            }
        },
        set(key, value) {
            try {
                area().setItem(key,
JSON.stringify(value));
                return true;
            } catch {
                return false;
            }
        },
        remove(key) {
            try { area().removeItem(key); } catch { /*
nothing to clean up */ }
        }
    };
};

export const local = wrap('localStorage');
export const session = wrap('sessionStorage');
```

Storage can throw (private browsing, blocked cookies), so every call is wrapped and fails soft. `wrap` is a factory: each object it returns remembers its storage `name` in a closure. Values go through `JSON.stringify` / `parse`.

Web Storage · Closures · JSON

utils.js · lines 6-12

```
export const debounce = (fn, wait = 200) => {
    let timer;
    return (...args) => {
        clearTimeout(timer);
        timer = setTimeout(() => fn(...args), wait);
    };
};
```

`debounce` returns a new function that waits until typing stops. `timer` lives in the closure, so each debounced function has its own. It's used for search (160 ms) and the draft autosave (400 ms).

Scope · Closures · Higher-order

6. Form validation rules

contact-form.js · lines 10–36

```
const NAME_RE = /^\p{L}[\p{L}\p{M} .'-]{1,59}$/u;
// Deliberately loose: something@something.tld, no
spaces. The mail app does the real check.
const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[a-z]{2,}$/i;
const LINK_RE = /https?:\/\/\S+/gi;

const RULES = {
    user_name(value) {
        if (!value) return 'Tell me what to call you.';
        if (value.length < 2) return 'That looks a
little short for a name.';
        if (!NAME_RE.test(value)) return 'Letters,
spaces, dots and hyphens only.';
        return '';
    },
    user_email(value) {
        if (!value) return 'I need an email to reply
to.';
        if (!EMAIL_RE.test(value)) return 'That email
doesn\'t look complete. Something like
you@example.com.';
        return '';
    },
    reason(value) {
        return value ? '' : 'Pick the closest one, it
helps me reply faster.';
    },
    message(value) {
        if (value.length < 20) return `A little more,
please. At least 20 characters (${20 - value.length} to
go).`;
        if (value.length > MESSAGE_MAX) return `Keep it
under ${MESSAGE_MAX} characters.`;
        if ((value.match(LINK_RE) ?? []).length > 2)
return 'Two links at most. Anything more and it reads
like spam.';
        return '';
    }
};
```

Each field has a function that returns an error message, or an empty string when the value is fine. `\p{L}` with the `u` flag accepts letters from any language, so names like "Zoë" or Hindi names pass.

Regex · Form validation · Objects

contact-form.js · lines 48–55

```
function check(field) {
    const message = RULES[field.name](valueOf(field));
    const error =
document.getElementById(field.getAttribute('aria-
describedby')?.split(' ')[0]);
    field.setAttribute('aria-invalid',
String(Boolean(message)));
    field.closest('.field')?.classList.toggle('has-
error', Boolean(message));
    if (error) error.textContent = message;
    return !message;
}
```

Shows or clears the message and marks the field with `aria-invalid`, so screen readers announce it. The error element is found through the field's `aria-describedby`.

DOM · Accessibility

7. Dates, time zones, Geolocation and audio

desk.js · lines 11–18

```
const OFFSET_RE = /GMT(?:([+-])(\d{1,2})(?::
(\d{2}))?)?/;
function offsetMinutes(timeZone, date = new Date()) {
    const name = new Intl.DateTimeFormat('en-US', {
timeZone, timeZoneName: 'shortOffset' })
        .formatToParts(date)
        .find(part => part.type ===
'timeZoneName')?.value ?? 'GMT';
    const [, sign = '+', hours = 0, minutes = 0] =
name.match(OFFSET_RE) ?? [];
    return (sign === '-' ? -1 : 1) * (Number(hours) * 60
+ Number(minutes));
}
```

`Intl` formats the time-zone name as "GMT+5:30". The regex pulls out the sign, hours and minutes, and the function returns minutes east of UTC. The difference between two zones gives "I'm 4.5 hours ahead of you".

`Date and time` `Regex` `Destructuring`

desk.js · lines 35–42

```
// Great-circle distance in km (haversine)
function distanceKm(a, b) {
    const rad = (deg) => (deg * Math.PI) / 180;
    const dLat = rad(b.lat - a.lat);
    const dLon = rad(b.lon - a.lon);
    const h = Math.sin(dLat / 2) ** 2 +
Math.cos(rad(a.lat)) * Math.cos(rad(b.lat)) *
Math.sin(dLon / 2) ** 2;
    return 2 * 6371 * Math.asin(Math.sqrt(h));
}
```

desk.js · lines 44–50

```
const locate = (options) => new Promise((resolve,
reject) => {
    if (!('geolocation' in navigator)) {
        reject(Object.assign(new
Error('unsupported'), { code: 0 }));
        return;
    }

navigator.geolocation.getCurrentPosition(resolve,
reject, options);
});
```

`locate` turns the callback-style Geolocation API into a Promise, so it can be `await`ed. `distanceKm` is the haversine formula, using `Math` and the `**` operator.

desk.js · lines 123–145

```
audio.addEventListener('loadedmetadata', () => {
    player.hidden = false;
    seek.max = audio.duration;
    time.textContent = format(audio.duration);
});
audio.lastElementChild?.addEventListener('error', () =>
{ player.hidden = true; });

audio.addEventListener('timeupdate', () => {
    seek.value = audio.currentTime;
    seek.style.setProperty('--progress',
`${(audio.currentTime / audio.duration) * 100 || 0}%`);
    time.textContent = format(audio.duration -
audio.currentTime);
});
audio.addEventListener('play', () => setPlaying(true));
audio.addEventListener('pause', () =>
setPlaying(false));
audio.addEventListener('ended', () => {
    audio.currentTime = 0;
    time.textContent = format(audio.duration);
});

toggle.addEventListener('click', () => {
    audio.paused ? audio.play().catch(() =>
setPlaying(false)) : audio.pause();
});
seek.addEventListener('input', () => { audio.currentTime
= Number(seek.value); });
```

The player is built entirely on media events. `loadedmetadata` shows it and sets the length, `timeupdate` moves the seek bar through a CSS variable, and `play` / `pause` / `ended` switch the button.

Audio events · CSS variables from JS

8. Organisation, performance and safety

app.js · lines 14-26

```
const repoSection = document.getElementById('repos');
if (repoSection) {
    debug.guard('repo-index', () => {
        const index = initRepoIndex(repoSection);
        // Don't spend one of GitHub's 60 hourly
requests until the section is close
        const watcher = new IntersectionObserver(entries
=> {
            if (!entries.some(entry =>
entry.isIntersecting)) return;
            watcher.disconnect();
            index.load();
        }, { rootMargin: '600px 0px' });
        watcher.observe(repoSection);
    });
}
```

The entry module starts each feature through `guard`, so a crash in one feature is logged and the rest keep working. The repo index waits for an `IntersectionObserver`, which saves GitHub's 60 requests per hour for visitors who actually scroll there.

debug.js · lines 27-35

```
export const guard = (name, init) => {
    try {
        time(name);
        init();
        time(name, true);
    } catch (err) {
        warn(`${name} failed to start`, err);
    }
};
```

9. HTML: SEO, table and template

index.html · lines 7-17

```
<meta name="description" content="Lukshya Supyal, B.Tech
CSE student at Lovely Professional University. I build
machine learning models and Python tools, and fix bugs
in the open-source libraries I use.">
<meta name="author" content="Lukshya Supyal">
<meta name="theme-color" content="#0a0a0a">
<link rel="canonical"
href="https://lushadowx.github.io/portfolio2/">
<link rel="icon" href="assets/images/favicon.svg"
type="image/svg+xml">

<meta property="og:type" content="website">
<meta property="og:site_name" content="Lukshya Supyal">
<meta property="og:title" content="Lukshya Supyal |
Ideas into reality">
<meta property="og:description" content="Machine
learning, Python tools and open-source fixes. Projects,
certificates and everything I've pushed to GitHub.">
<meta property="og:url"
content="https://lushadowx.github.io/portfolio2/">
```

The meta description and canonical URL tell search engines what the page is and where it lives. Open Graph tags control the preview card when the link is shared.

index.html · lines 321-330

```
<table class="repo-table" id="repo-table"
hidden>
    <caption class="sr-only">My GitHub
repositories. Column headers sort the table.
</caption>
    <thead>
        <tr>
            <th scope="col" data-sort="name"
aria-sort="none"><button
type="button">Name</button></th>
            <th scope="col" data-sort="language"
aria-sort="none"><button
type="button">Language</button></th>
            <th scope="col" data-sort="stars"
aria-sort="none"><button
type="button">Stars</button></th>
            <th scope="col" data-sort="pushed"
aria-sort="none"><button
type="button">Updated</button></th>
        </tr>
    </thead>
```

index.html · lines 335-345

```
<template id="repo-card-tpl">
    <li class="repo-card" draggable="true">
        <article>
            <p class="repo-card__meta"><span
class="repo-card__lang"></span><span class="repo-
card__stars"></span></p>
            <h3 class="repo-card__name"><a
target="_blank" rel="noopener"></a></h3>
            <p class="repo-card__desc"></p>
            <ul class="repo-card__topics" aria-
label="Topics"></ul>
            <footer class="repo-card__foot"><time>
</time><button type="button" class="repo-pin" aria-
pressed="false">Pin</button></footer>
        </article>
    </li>
</template>
```

A `<template>` isn't rendered. JavaScript clones it for each repository and fills in the text with `textContent`, which never runs code, so a description with HTML in it can't inject anything.

HTML5 DOM Security

10. CSS: variables, container query, custom controls, animation

repo-index.css · lines 6-21

```
.repo-index {
    --ink: #11110f;
    --paper: #f2f0e9;
    --card: #f8f6f1;
    --muted: #6e6a61;
    --faint: #8e897e;
    --line: #11110f24;
    --line-dark: var(--line);
    --accent: #c15f3c;
    --danger: #a33a1c;
    --mono: 'SFMono-Regular', Consolas, 'Liberation
Mono', monospace;
    position: relative; z-index: 10; padding: 120px 0
56px;
    background: var(--paper); color: var(--ink);
    font-family: 'Inter', ui-sans-serif, -apple-system,
BlinkMacSystemFont, 'Segoe UI', sans-serif;
    -webkit-font-smoothing: antialiased;
}
```

repo-index.css · lines 235-242

```
@container (max-width: 560px) {
    .repo-table thead { position: absolute;
width: 1px; height: 1px; overflow: hidden; clip-
path: inset(50%); }
    .repo-table tr { display: grid; grid-
template-columns: 1fr auto; gap: 6px 12px;
padding: 14px 0; border-bottom: 1px solid var(--
line); }
    .repo-table th, .repo-table td { padding: 0;
border: 0; }
    .repo-table tbody th { grid-column: 1 / -1;
}
    .repo-table td[data-label="Stars"]::before {
content: '\2605\00a0'; color: var(--faint); }
    .repo-table td[data-label="Updated"] { grid-
column: 1 / -1; font-size: 12px; }
}
```

repo-index.css · lines 71-83

```
/* switch-style checkbox */
.repo-check { display: flex; align-items: center; gap:
10px; padding-bottom: 12px; cursor: pointer; white-
space: nowrap; }
.repo-controls .repo-check > span { margin: 0; color:
var(--ink); font: 600 13px/1 'Inter', sans-serif;
letter-spacing: 0; text-transform: none; }
.repo-check input {
    position: relative; flex: none; width: 34px; height:
20px; margin: 0; cursor: pointer; appearance: none;
    background: #11110f26; border-radius: 99px;
transition: background-color 0.2s;
}
.repo-check input::before {
    content: ''; position: absolute; top: 3px; left:
3px; width: 14px; height: 14px; border-radius: 50%;
    background: #fff; box-shadow: 0 1px 2px #11110f40;
transition: transform 0.25s var(--ease-out-expo);
}
.repo-check input:checked { background: var(--ink); }
.repo-check input:checked::before { transform:
translateX(14px); }
```

repo-index.css · lines 204-209

```
.repo-card--ghost {
    height: 196px; border-radius: 14px;
    background: linear-gradient(100deg,
#11110f0a 30%, #11110f17 50%, #11110f0a 70%) 0 0
/ 300% 100%;
    animation: repo-shimmer 1.3s linear
infinite;
}
@keyframes repo-shimmer { from { background-
position: 100% 0; } to { background-position: 0
0; } }
```

09 · TESTING AND DEBUGGING

How I checked it works

I tested the site in Chrome on desktop (1440 px) and in phone mode (390 px), and with network requests to GitHub blocked to force the fallback and error paths.

#	TEST	EXPECTED	RESULT
1	Open the repo index normally	Cards load. Status reads "Live from the GitHub API".	Pass
2	GitHub returns 403 (rate limit)	Falls back to `repos.json` and says so (Fig 29)	Pass
3	API and snapshot both blocked	Error message with a "try again" button (Fig 30)	Pass
4	Search `lang:python stars:>0`	3 results	Pass
5	Search `stars:abc`	Message explains the filter couldn't be read	Pass
6	Pin by button, by drag and with arrow keys, then reload	Order is kept after the reload	Pass
7	Pin a 7th repo	Shelf shakes and says it holds 6	Pass
8	Click the Stars header twice	Sorts descending, then ascending. `aria-sort` updates.	Pass
9	Submit an empty contact form	4 messages, focus on Name (Fig 28)	Pass
10	Email `asha@example`	"That email doesn't look complete..."	Pass
11	Type in the form, then reload	Draft is restored with "Start over" (Fig 19)	Pass
12	Visitor in London, location set to Delhi	"4.5 hours ahead", "about 328 km" (Fig 20)	Pass
13	Deny location permission	"No problem, location stays with you."	Pass
14	Phone width (390 px)	No sideways scrolling. One-column controls. Stacked table.	Pass
15	Keyboard only (Tab)	Skip link first, visible focus everywhere, every action reachable	Pass
16	Repos with HTML in the description	Shown as text, never run (`textContent` / `escapeHTML`)	Pass

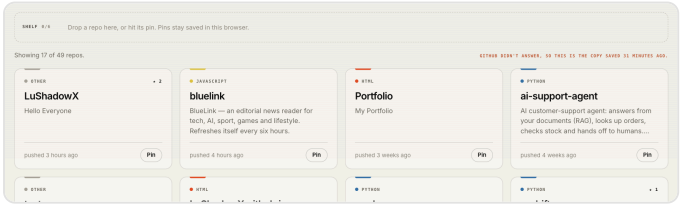


Fig. 29 · **Fallback.** GitHub's API blocked, so the saved copy is shown with an orange notice.



Fig. 30 · **Error state.** Nothing reachable: a clear message and a retry link instead of a blank area.

Debug mode

Opening `lushadowx.github.io/portfolio2/?debug` turns on extra console output: how long each feature took to start (`performance.mark` / `measure`), a `console.table` of every repository with its language, stars and fork flag, and warnings with the full error when a fallback happens. Without `?debug` the console stays clean.

10 · DEPLOYMENT

Hosted on GitHub Pages

The code lives in a public GitHub repository, and GitHub Pages serves it as a website straight from the `main` branch.

1. Created the repository `LuShadowX/portfolio2` and pushed the code.
2. Settings → Pages → Source: **Deploy from a branch**, branch `main`, folder `/ (root)`.
3. GitHub builds and publishes to **https://lushadowx.github.io/portfolio2/**. Every push redeploys automatically.
4. Added the website link and description to the repository, and a README explaining features, structure and how to run it locally.

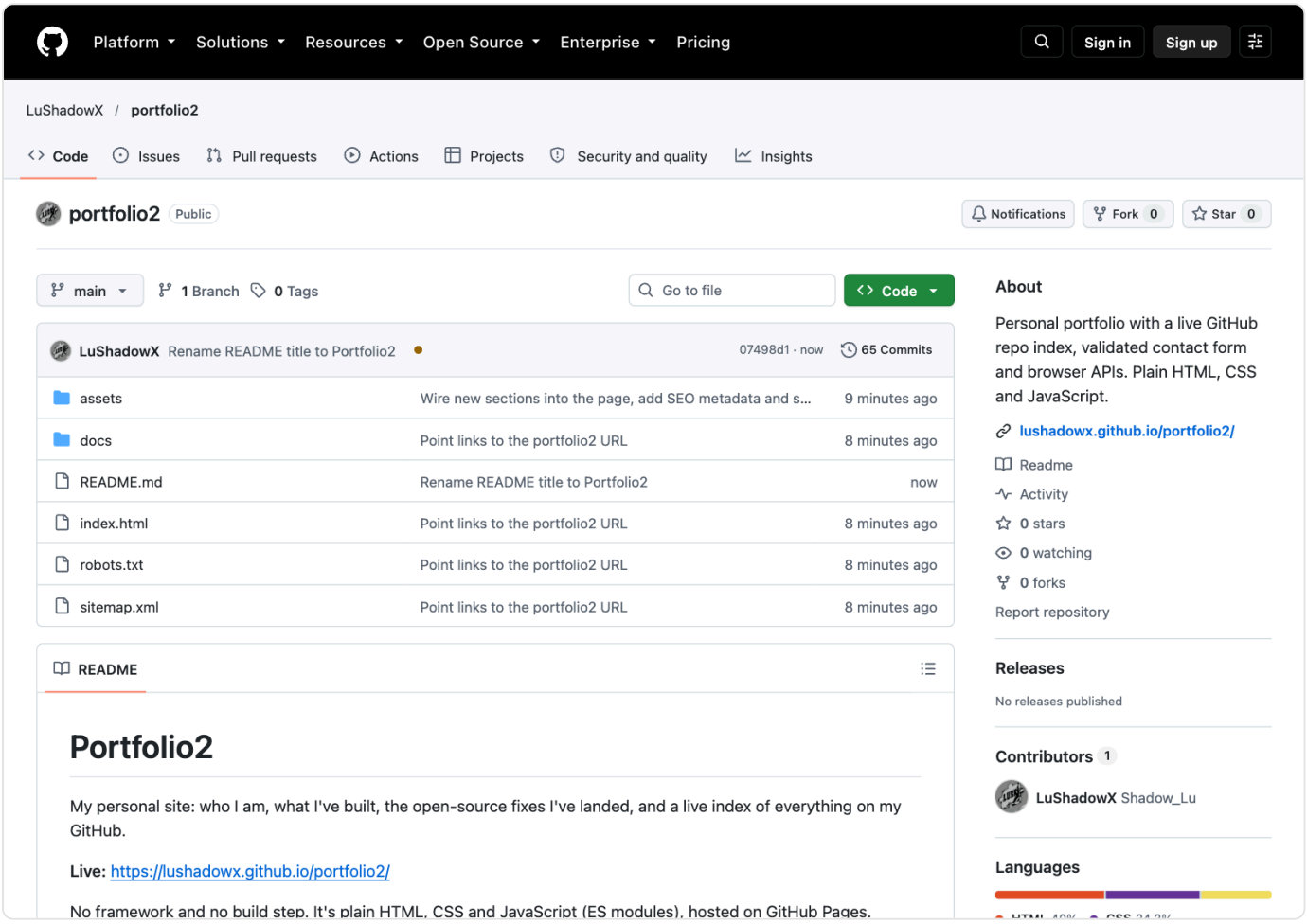


Fig. 31 · **The repository**, with folders, README, the website link and a successful GitHub Pages deployment.

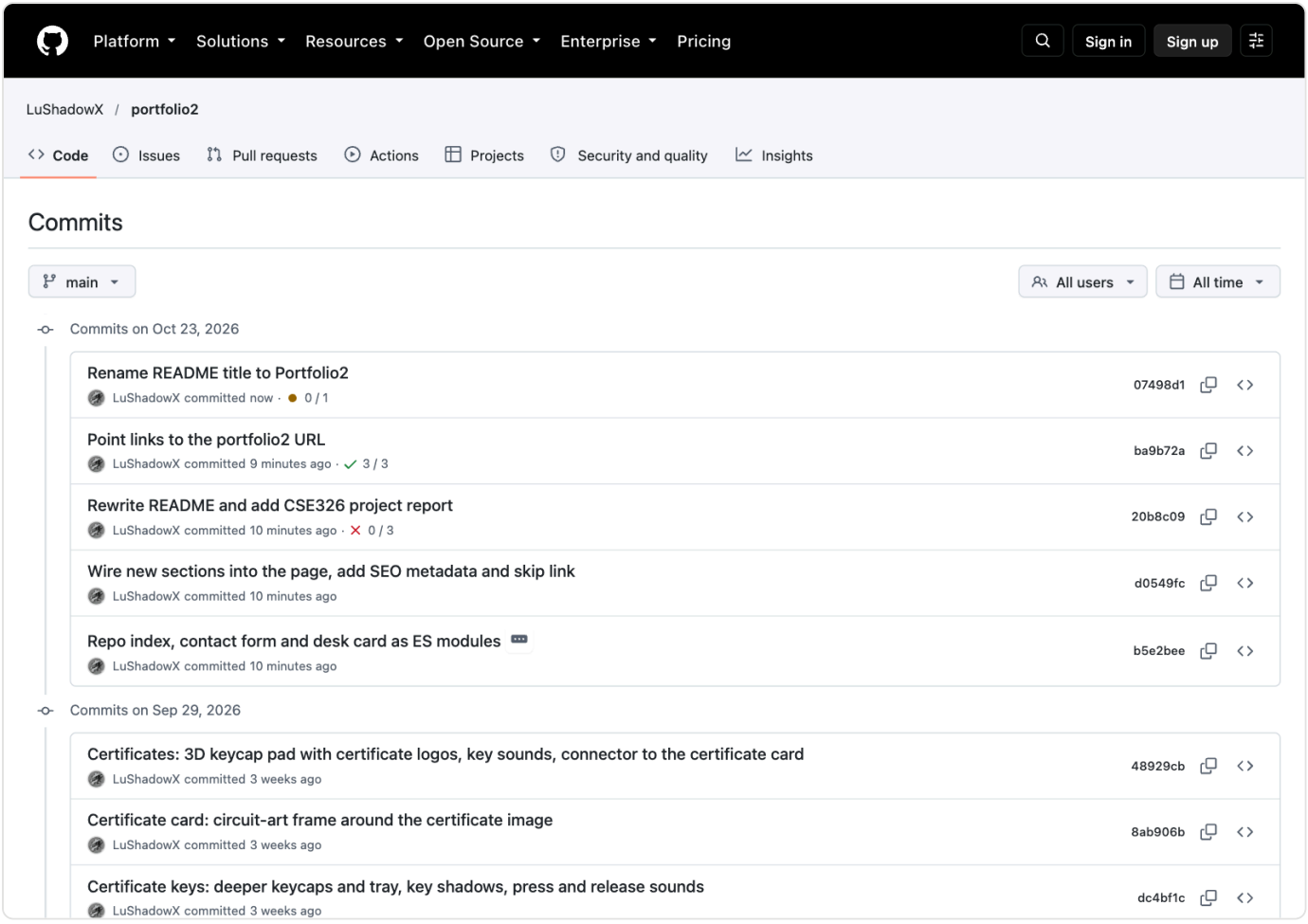


Fig. 32 · **Commit history.** Changes are split into small commits with clear messages.

Running it locally

```
cd portfolio2
python3 -m http.server 8000
# open http://localhost:8000   (ES modules need a server, not file://)
```

11 · CONCLUSION

What I learned

Portfolio2 puts every part of CSE326 into one site that people actually use, rather than ten separate lab pages.

- **HTML is the foundation for accessibility and SEO.** Choosing `table`, `fieldset`, `time` and proper labels made the page work for screen readers and search engines with almost no extra code.
- **Modern CSS can do layout jobs that once needed JavaScript.** The container query restacks the table based on the space it actually has, not just the screen width.
- **Real data is messy.** APIs hit rate limits, users go offline, and storage can be blocked. Planning for failure (cache → API → snapshot, safe storage, `guard`) is what makes an app feel reliable.
- **Small, well-named modules are easier to debug.** When something broke, the `?debug` output pointed to the exact module.
- **Browser APIs are powerful but need consent and care.** Location is requested only on click, never stored, and every error case has a friendly message.

Possible next steps

- A light/dark theme switch saved in localStorage
- A service worker so the whole site works offline
- Recording and adding the voice note

> **Links**
> Live website: https://lushadowx.github.io/portfolio2/
> Source code: https://github.com/LuShadowX/portfolio2